

From alert to answer.

An automated root-cause analyst for ad-tech metrics — detects when revenue, fill rate, eCPM, or CTR breaks its seasonal baseline, drills down in ClickHouse to the exact segment responsible, and narrates a diagnosis where every number is reproducible from the data.

TRACK

Click-a-thon 2026 · InMobi

STACK

ClickHouse Cloud · Next.js · Langfuse · eve agents

"Did it move?" is answered.
"Why?" still takes a human hours.



ClickHouse does the analysis. The LLM only narrates it.

Every number in a Sentinel diagnosis is the output of a ClickHouse query, not a language model guess. The model's only job is to read query results and choose which query to run next — it never computes a figure itself.

Three trust levels, kept separate

Detection is deterministic and reproducible. Investigation is agentic but evidence-gated. Remediation is advisory and never mixed into the evidence-backed diagnosis. Each stage has its own failure mode — and its own place in the trace.

Built for the unseen incident

Nothing is tuned to the anomalies we can see in the dev dataset. Thresholds stay statistical — z-score against a same-weekday/same-hour seasonal baseline — because the system is graded on a fresh incident released on day two.



How Sentinel works, end to end.

Four stages, four different jobs. Each hands off a concrete artifact to the next — an anomaly row, a verdict, a rendered page — never a vague hand-wave.

01

Ingest

~9M ad events over 5 weeks land in ClickHouse as a star schema — `ad_events` joined to apps, advertisers, and geo/device dimensions.

02

Detect

Materialized views recompute seasonal z-scores on a schedule, global and per-segment. No LLM. Flags land in `anomalies / segment_anomalies`.

03

Investigate

A subagent picks up one flagged incident, queries ClickHouse for evidence, ranks candidate segments, and returns a structured, cited verdict.

04

Present

The Next.js dashboard renders the incident, its chart, and its evidence — plus a chat panel to ask follow-up questions against live ClickHouse data.

`ad_events` → `anomalies` → `verdict` → `dashboard` — deterministic compute stays upstream of anything agentic



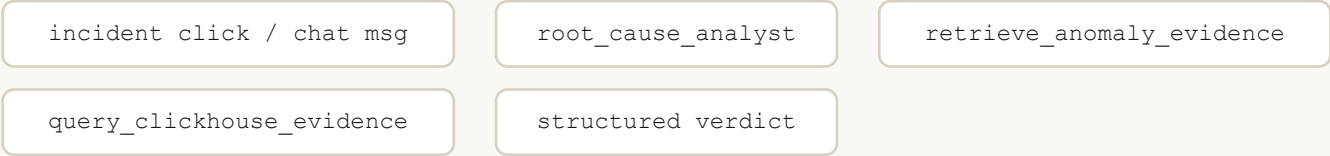
One pipeline, three services, real file names.

Not a diagram of intent — this is what actually runs, traced through apps/detection-service, apps/sentinel-agent, and apps/web.

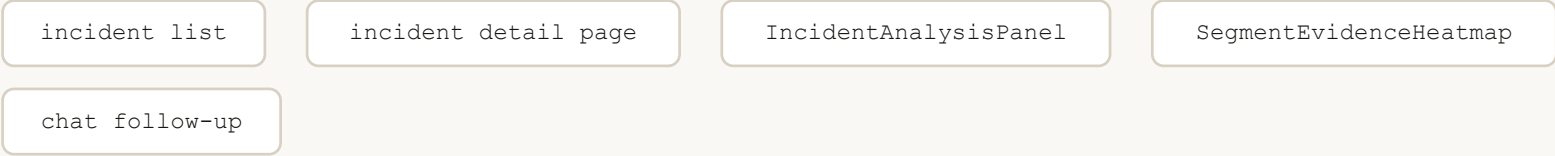
DETECT · CLICKHOUSE



INVESTIGATE · SENTINEL-AGENT



PRESENT · APPS/WEB



10-minute refreshable materialized views own detection's lifecycle end to end — nothing in the other two lanes runs until a row actually lands in incidents.



One seasonal baseline. Three independent checks.

Every hour of every metric is compared to its own trailing 4-week, same-weekday, same-hour average — never a flat global mean. The residual is shrunk toward a pooled noise floor so sparse segments don't produce wild swings.

SQL/MV/02 — SEASONAL RESIDUAL

```
-- trailing 4-week, same (dow, hod) baseline
avg(value) OVER w AS mean_v,
WINDOW w AS (PARTITION BY metric, dow, hod
ORDER BY d ROWS BETWEEN 4 PRECEDING AND 1 PRECEDING)
-- shrink toward the pooled noise floor
(value - mean_v) / sqrt((n*var_v + 3*pooled_var)/(n+3)) AS zr
```

SQL/MV/03 — FLAG WHEN IT CROSSES

```
SELECT metric, hour_ts, zr AS z,
'trend_seasonal' AS method
FROM metric_zr_hourly
-- z_threshold is config, not a hand-fit constant
WHERE abs(zr) > z_threshold
```

trend_seasonal — scores every hourly residual directly against `z_threshold`

proportion — two-proportion z-test for rate metrics (fill rate, render rate, CTR)

day_level — one-sample t-test pooling a completed day's 24 residuals

Why segment-level, too

Confirmed on our dev data: native eCPM rose ~7% while interstitial fell ~9% over 3 days — global daily eCPM stayed flat at 2.475 the whole time. The same ensemble runs one level deeper, across 46 segment values.

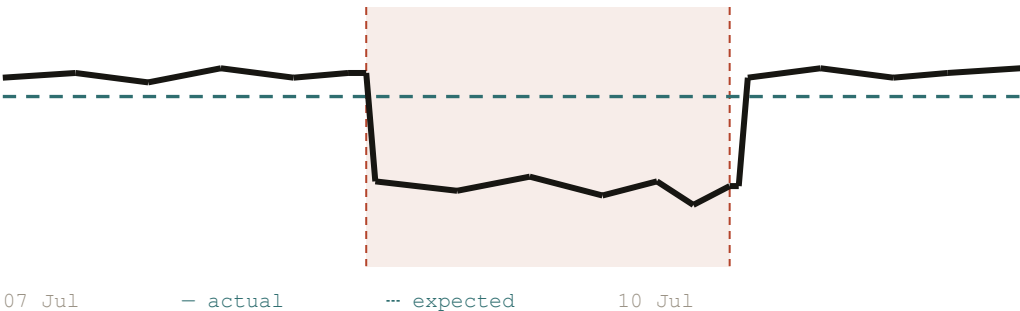


One real incident, start to finish.

DETECTED

Fill rate fell below its seasonal baseline.

Fill rate fell **7.0%** from its same-hour seasonal baseline, 08 Jul 00:00 – 09 Jul 23:00 UTC. Peak movement 77.71% → 72.30%.



ID · country	z 6.97
APAC · region	z 6.51
utility · category	z 6.31

Why it fired

Every hour in the 48-hour window was flagged, confirmed by 3 independent methods (day_level, proportion, trend_seasonal) — the strongest hour reached seasonal |z| 3.91. Persistence plus method agreement is the reportability bar.

THE QUERY — PROPORTION METHOD, SQL/MV/03

```
-- fill_rate = fills / requests, trailing baseline
(observed_rate - p0) / sqrt(p0*(1-p0)/den) AS z
-- guard: degenerate when a sparse cell's
-- baseline rate is exactly 0 or 1
WHERE p0 > 0 AND p0 < 1 AND den > 0
      AND abs(z) > z_threshold
```

What actually called it

retrieve_anomaly_evidence — first tool call, always. Reads incidents, anomalies, segment_anomalies, metrics_hourly_v for this exact scope. No dashboard values, no mock data.



The same incident — what got ruled out.

Every ad_format co-moved with roughly equal strength on this incident. Uniform movement rules that dimension out as the differentiator — the driver sits elsewhere (country/region, whose z-scores run higher and don't apply uniformly).

ad_format	Peak z	Mean z	Incident corr.	Evidence
native	-5.04	-2.76	100%	supports, but uniform
interstitial	-4.92	-2.39	100%	supports, but uniform
video	-4.56	-2.43	100%	supports, but uniform
banner	-4.10	-2.73	100%	supports, but uniform
rewarded	-3.85	-2.40	100%	supports, but uniform

"No supporting movement" — seasonal $|z| < 1$ across $\geq 75\%$ of scored hours — rules a segment out statistically. It never proves causality; a correlation still needs a second confirming query before the verdict calls it causal.

No fabricated numbers

Every cited figure traces back to a specific ClickHouse row. The agent narrates results; it never invents a percentage.

Langfuse trace

Every tool call the investigator makes — which query, on what scope, in what order — is captured via @langfuse/otel.

Audit-trail tables

anomalies and segment_anomalies are never truncated — a reprocessed hour replaces its row, so full history stays queryable.



ClickHouse computes. Next.js shows it.

CLICKHOUSE CLOUD	Primary datastore and analytical engine — every drill-down runs as a materialized view or a bounded query, never as raw rows streamed into an LLM.
NEXT.JS	The visualization layer (apps/web) — the live incident feed, segment evidence heatmap, and chat panel all render straight off ClickHouse, no separate BI tool.
LANGFUSE	LLM observability for the investigation loop — the required trace that proves the diagnosis, not the dashboard, produced the answer.
CLICKSTACK / HYPERDX	Browser + service telemetry for the dashboard and detection service, via a local OTel collector.
EVE AGENT FRAMEWORK	Runs the investigator as a subagent with two scoped, read-only ClickHouse tools and Slack + chat channels.





CLOSE

Fast, localized, honest — because the compute is real.

Sentinel doesn't ask an LLM to guess why revenue moved. It asks ClickHouse, every time, and only lets the model describe what ClickHouse already proved. That's the whole bet — and it's the only way an evidence-backed diagnosis survives an unseen incident.